

```
OneCompiler
commands.sql x +
1  /* atividade aula 12 - Revisada */
2
3  DROP TABLE IF EXISTS carro, pessoa;
4
5  CREATE TABLE IF NOT EXISTS pessoa (
6  id_pessoa INTEGER PRIMARY KEY,
7  nome VARCHAR(100) NOT NULL,
8  nascimento DATE
9  );
10
11 CREATE TABLE IF NOT EXISTS carro (
12 id_carro INTEGER PRIMARY KEY,
13 placa CHAR(7) NOT NULL,
14 ano INTEGER,
15 id_pessoa INTEGER NOT NULL,
16 FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa) ON DELETE CASCADE
17 );
18
19 -- Dados de exemplo para rodar no OneCompiler
20 INSERT INTO pessoa (id_pessoa, nome, nascimento) VALUES
21 (1, 'Ana Silva', '1995-03-10'),
22 (2, 'João Santos', '1988-07-22'),
23 (3, 'Maria Souza', '2001-11-05'),
24 (4, 'Pedro Lima', '1979-01-30'),
25 (5, 'Carla Oliveira', '1992-09-14');
26
27 INSERT INTO carro (id_carro, placa, ano, id_pessoa) VALUES
28 (1, 'ABC1234', 2018, 1),
29 (2, 'DEF5678', 2020, 1),
30 (3, 'GHI9012', 2016, 2),
31 (4, 'JKL3456', 2021, 3),
32 (5, 'MNO7890', 2015, 4),
33 (6, 'PQR2468', 2019, 5);
34
35 SELECT * FROM carro;
```

Output:

```
DROP TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 5
INSERT 0 6
```

id_carro	placa	ano	id_pessoa
1	ABC1234	2018	1
2	DEF5678	2020	1
3	GHI9012	2016	2
4	JKL3456	2021	3
5	MNO7890	2015	4
6	PQR2468	2019	5

(6 rows)

```
indexname | indexdef
-----
pessoa_pkey | CREATE UNIQUE INDEX pessoa_pkey ON public.pessoa USING btree
(1 row)
```

QUERY PLAN

```
Seq Scan on pessoa (cost=0.00..14.00 rows=2 width=226) (actual time=0.004
Filter: ((nome)::text = 'Ana Silva')::text)
Rows Removed by Filter: 4
```

```
/* atividade aula 12 - Revisada */
```

```
DROP TABLE IF EXISTS carro, pessoa;
```

```
CREATE TABLE IF NOT EXISTS pessoa (
  id_pessoa INTEGER PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  nascimento DATE
);
```

```
CREATE TABLE IF NOT EXISTS carro (
  id_carro INTEGER PRIMARY KEY,
  placa CHAR(7) NOT NULL,
  ano INTEGER,
  id_pessoa INTEGER NOT NULL,
  FOREIGN KEY (id_pessoa) REFERENCES pessoa(id_pessoa) ON DELETE CASCADE
);
```

```
-- Dados de exemplo para rodar no OneCompiler
```

```
INSERT INTO pessoa (id_pessoa, nome, nascimento) VALUES
(1, 'Ana Silva', '1995-03-10'),
(2, 'João Santos', '1988-07-22'),
(3, 'Maria Souza', '2001-11-05'),
(4, 'Pedro Lima', '1979-01-30'),
(5, 'Carla Oliveira', '1992-09-14');
```

```
INSERT INTO carro (id_carro, placa, ano, id_pessoa) VALUES
(1, 'ABC1234', 2018, 1),
```

```
(2, 'DEF5678', 2020, 1),
(3, 'GHI9012', 2016, 2),
(4, 'JKL3456', 2021, 3),
(5, 'MNO7890', 2015, 4),
(6, 'PQR2468', 2019, 5);
```

```
SELECT * FROM carro;
```

```
/* Comando para listar os índices de uma tabela: */
```

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';
```

```
/* Parte A - Compreender a diferença entre Seq Scan e Index Scan */
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa WHERE nome = 'Ana Silva';
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa WHERE nome = 'João Santos';
```

```
/* Crie um índice B-tree na coluna nome: */
```

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```

```
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
```

```
/* Parte C - Execute novamente as duas consultas */
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa WHERE nome = 'Ana Silva';
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa WHERE nome = 'João Santos';
```

```
/*
```

Parte D - Respostas Objetivas:

- Qual plano foi utilizado antes do índice?

Resposta: Foi utilizado o plano de Varredura Sequencial (Seq Scan).

- Qual plano foi utilizado após o índice?

Resposta: Foi utilizado o plano de Busca por Índice (Index Scan ou Bitmap Index Scan).

- Houve redução no tempo de execução? O índice foi utilizado em ambos os nomes? Justifique.

Resposta: Sim, houve redução. O índice foi utilizado porque as consultas são seletivas.

- Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

Resposta: Porque o custo de ler a tabela inteira pode ser menor em tabelas pequenas ou em consultas pouco seletivas.

*/

/* Exercício 3 - Índice Composto e Ordem das Colunas */

/* Exercício 3 - PARTE A */

```
DROP INDEX IF EXISTS idx_pessoa_nascimento;
```

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa
```

```
WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';
```

/* Exercício 3 - PARTE B */

```
CREATE INDEX idx_pessoa_nascimento_nome ON pessoa (nascimento, nome);
```

/* Exercício 3 - PARTE C e D */

```
EXPLAIN ANALYZE
```

```
SELECT * FROM pessoa WHERE nome = 'Ana Silva';
```

/*

Exercício 3 - Respostas:

- Qual plano foi utilizado antes da criação do índice composto?

Resposta: Seq Scan.

- Qual plano foi utilizado após a criação do índice composto?

Resposta: Index Scan ou Bitmap Index Scan.

- O índice composto foi utilizado na consulta que filtra apenas por nome?

Resposta: Não necessariamente.

- Por que a ordem das colunas no índice composto é relevante?

Resposta: Porque o PostgreSQL aproveita melhor a coluna da esquerda do índice.

*/

```

/* Exercício 4 */
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
DROP INDEX IF EXISTS idx_pessoa_nascimento;
DROP INDEX IF EXISTS idx_pessoa_nome;

CREATE INDEX idx_pessoa_nascimento ON pessoa (nascimento);
CREATE INDEX idx_pessoa_nome ON pessoa (nome);

/* Exercício 4 - PARTE B */
EXPLAIN ANALYZE
SELECT * FROM pessoa
WHERE nascimento >= DATE '2000-01-01' AND nome = 'Ana Silva';

```

```

/*
Exercício 4 - Respostas:

```

- O PostgreSQL utilizou os dois índices simples na consulta?
Resposta: Pode usar BitmapAnd, dependendo do plano.
 - Qual é o papel do operador BitmapAnd no plano de execução?
Resposta: Intersectar os resultados de dois índices.
 - O que o plano efetivamente fez?
Resposta: Pode usar um índice e aplicar o restante como filtro.
- ```

*/

```

```

/* EXERCÍCIO 5 */
EXPLAIN ANALYZE
SELECT * FROM carro WHERE ano BETWEEN 2015 AND 2020;

DROP INDEX IF EXISTS idx_carro_ano;
CREATE INDEX idx_carro_ano ON carro (ano);

```

```

/*
Resumo Exercício 5:

```

- Antes da Criação: Seq Scan.
- Depois da Criação: Index Scan ou Bitmap Index Scan.

```

*/

```

```

/* Exercício 6 */
DROP INDEX IF EXISTS idx_carro_ano;

```

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nome = 'Ana Silva';
```

```
DROP INDEX IF EXISTS idx_pessoa_nome;
DROP INDEX IF EXISTS idx_carro_id_pessoa;
```

-- Criação dos índices recomendados para otimização do JOIN e WHERE:

```
CREATE INDEX idx_pessoa_nome ON pessoa (nome);
CREATE INDEX idx_carro_id_pessoa ON carro (id_pessoa);
```

/\*

Análise do Exercício 6:

Resposta: O plano pode usar Nested Loop com Index Scan após os índices.

\*/

/\* EXERCÍCIO 7 \*/

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01' AND c.ano >= 2018;
```

/\* EXERCÍCIO 8 \*/

```
EXPLAIN ANALYZE
SELECT * FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```